

SE Radio · Episodio 662

Balancing Coupling in Software Design

Vlad Khononov — Arquitecto de software y autor

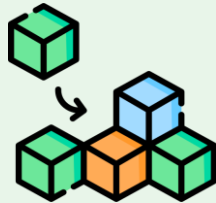
José Iván Díaz Potenciano

Enol de la Calle Iglesias

¿Qué es el Coupling?

Definición: cualquier dependencia entre dos componentes. El diseño de esa dependencia determina si un cambio en uno obliga a cambiar el otro.

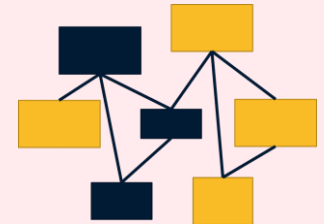
Modularidad



- Coupling bien diseñado
- Componentes independientes
- Cambios locales, sin cascada

VS

Complejidad



- Coupling mal diseñado
- Cambios en cadena inesperados
- Alta carga cognitiva

Modelos clásicos de Coupling

1979

Module Coupling

Yourdon & Constantine

7 tipos de coupling (data, stamp, control, external, common, content). Base teórica fundamental, pero pensada para sistemas de su época.

1992

Connascence

Meilir Page-Jones

Refinamiento del modelo: mide cuánto dos componentes 'nacen juntos'. Más expresivo, pero complejo de aplicar en microservicios modernos.

Hoy

El problema

Ambos modelos tienen ~50 años. Los sistemas actuales (microservicios, cloud, DDD) los desbordan. Necesitamos algo nuevo.

El framework: Balanced Coupling

El origen

- Vlad lideró un proyecto de microservicios que fracasó.
- Investigando el porqué, revisitó todos los modelos de coupling existentes.
- Conclusión: los modelos clásicos no se adaptan bien a los sistemas modernos.
- Así nació Balanced Coupling: unir 50 años de teoría en una sola herramienta simple.

"Añadir una feature no debería sentirse como desactivar una bomba."

— Vlad Khononov

Una fórmula + tres dimensiones = decisiones de diseño más claras

Las tres dimensiones del Coupling

1

Integration Strength

Fuerza de integración

¿Cuánto conocimiento comparte un componente del otro? A mayor conocimiento compartido → más fuerte el coupling.

Ejemplo: el servicio A conoce el esquema interno de la BD del servicio B → alto strength.

2

Distance

Distancia

¿Qué tan lejos están los componentes en la arquitectura? A mayor distancia → más caro propagar el cambio.

Ejemplo: dos módulos en el mismo servicio tienen distancia baja; dos microservicios distintos, alta.

3

Volatility

Volatilidad

¿Con qué frecuencia cambia un componente? Alta volatilidad + alto coupling = fuente de problemas.

Ejemplo: si el esquema de la BD cambia cada sprint → eso es volatilidad alta.

Aplicación práctica

Coupling problemático = Integration Strength alta + Distance grande + Volatility alta

Microservicios ≠ solución mágica

Muchos proyectos de microservicios acaban como 'monolitos distribuidos': servicios muy separados (distance alta) pero que comparten demasiado conocimiento (strength alta). El resultado es el peor escenario posible.

Relevante en la era de la IA

Si la IA puede generar código fácilmente, el diseño modular se convierte en el verdadero diferenciador. Balanced Coupling ayuda a evaluar decisiones de diseño —tanto de humanos como de IA— hacia la evolvabilidad a largo plazo.

Conclusión

- 01** El coupling siempre existirá — la clave es gestionarlo conscientemente, no eliminarlo.
- 02** Evalúa cada decisión de diseño desde las tres dimensiones: strength, distance y volatility.
- 03** Los modelos clásicos siguen siendo válidos, pero Balanced Coupling los actualiza para el mundo real de hoy.
- 04** Un buen diseño modular hace que añadir features sea sencillo, no un riesgo.

Turno de preguntas